

神经网络加速器计算核心设计

陈天乐 无 25

2026 年 5 月 23 日

目录

1	实验目的	2
2	基本任务	2
2.1	乘累加树型阵列设计与实现	2
2.2	脉动阵列型设计与实现	3
2.3	拓展脉动阵列为 $M \times N$ 设计与实现	5
3	思考题	7

1 实验目的

参考“加速器”、“映射”部分的课程内容，使用 Verilog/SystemVerilog 语言设计实现 CNN 加速器的核心加速模块——计算阵列。

2 基本任务

2.1 乘累加树型阵列设计与实现

本实验首先实现了一个 $1 \times N$ 乘累加树阵列，对应代码文件为 `mac_tree.sv`，测试文件为 `tb_mac_tree.sv`。阵列主要参数为 `MAC_NUM` 和 `BRANCH_NUM`。其中 `MAC_NUM` 表示并行输出 lane 的数量；`BRANCH_NUM` 表示每棵加法树在一个周期内接收的输入激活值和权重对数。取 $N = 10$ ，因此每个输出块一共需要 15 个计算周期完成累加。

该设计的前仿结果完全正确，总耗时为 25645ns，仿真结果如下：

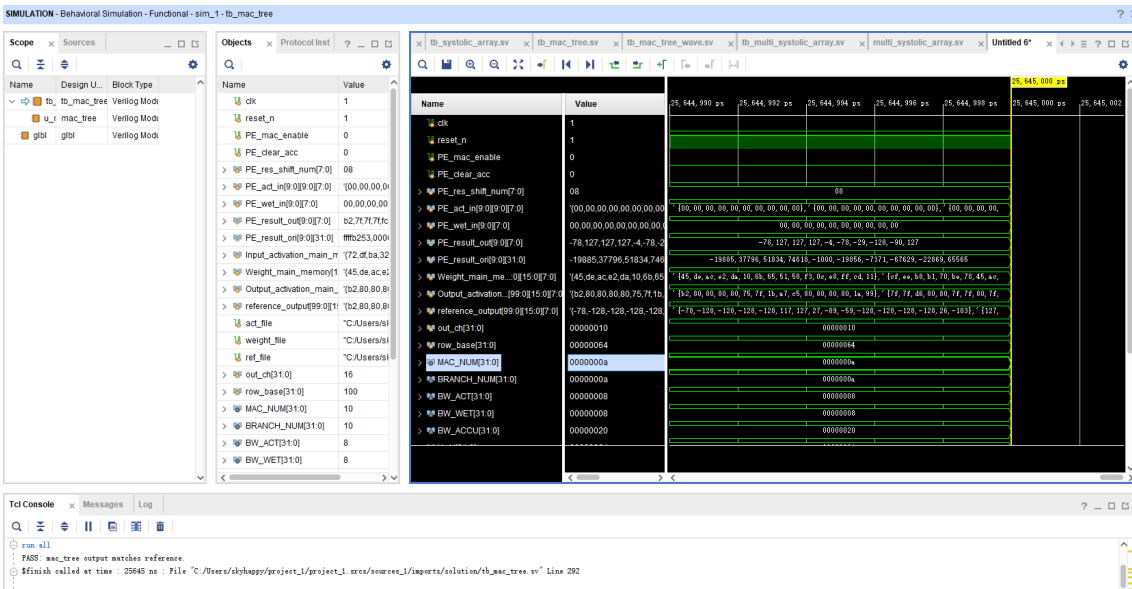


图 1: 1×10 乘累加树计算阵列仿真波形

```
1 mac_tree match_count=1600/1600, wrong_num=0
2 PASS: mac_tree all 1600 outputs match reference.
```

Listing 1: 乘累加树阵列仿真检查结果

乘累加树型阵列的资源利用如下：

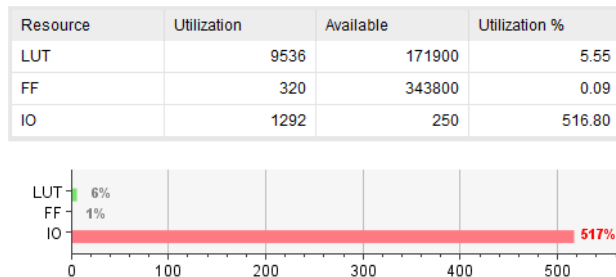


图 2: 1×10 乘累加树计算阵列资源利用报告

乘累加树型阵列的时序报告如下：

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 8.477 ns	Worst Hold Slack (WHS): 0.103 ns	Worst Pulse Width Slack (WPWS): 4.650 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 320	Total Number of Endpoints: 320	Total Number of Endpoints: 321

All user specified timing constraints are met.

图 3: 1*10 乘累加树计算阵列时序报告

通过时序报告可以看到, 时序约束符合要求, 并且在 100MHz 的时钟下, WNS 为 8.477ns, 说明可以用更快的时钟运行, 并且提升空间很高。

表 1: 乘累加树阵列在 100MHz 下的设计参数

算力	$10 \times 10 \times 100M = 10Gops$
计算量/算力	$100 \times 150 \times 16 / 10Gops = 24us$
最高时钟频率	$\frac{1}{10ns - 8.477ns} = 656.60MHz$
阵列利用率	$24 / 25.645 = 93.59\%$
IA 带宽需求	$8b \times 10 \times 10 \times 100MHz = 80Gbps$
W 带宽需求	$8b \times 10 \times 100MHz = 8Gbps$
OA 带宽需求	$8b \times 10 \times 100MHz = 8Gbps$
IA 片上 buffer 容量	$8b \times 10 \times 10 = 800bit$
W 片上 buffer 容量	$8b \times 10 = 80bit$
OA 片上 buffer 容量	$8b \times 10 = 80bit$
IA 片外访问总 bit 数	$16 \times 100 \times 150 \times 8bit$
OA 片外写回总 bit 数	$16 \times 10 \times 150 \times 8bit$
W 片外访问总 bit 数	$16 \times 100 \times 8bit$

2.2 脉动阵列型设计与实现

第二个实验实现了 $1 \times N$ 脉动阵列, 对应代码文件为 `systolic_array.sv`, 测试文件为 `tb_systolic_array.sv`。我设计的脉动阵列的 `CELL_H=10`, `CELL_W=1`。阵列中 10 个 PE 纵向连接, 采用 `weight-stationary` 策略, 权重在开始时加载到 PE 内部并保持不变, 激活值按照斜向时序输入, 部分和沿阵列方向逐级向下传递。这样右侧或下游 PE 的乘加结果会比上游晚一个周期出现, 符合脉动阵列数据逐拍传播的特征。需要指出的是, 第 0 个 PE 接收外部部分和 `PE_partial_sum`, 后续 PE 接收上一级 PE 的结果, 形成一条 10 级流水。仿真检查结果如下:

该设计的前仿结果完全正确, 总耗时为 266445ns, 主要开销来自 15 个 tile 需要依次流过整条 10 级脉动链。仿真结果如下:

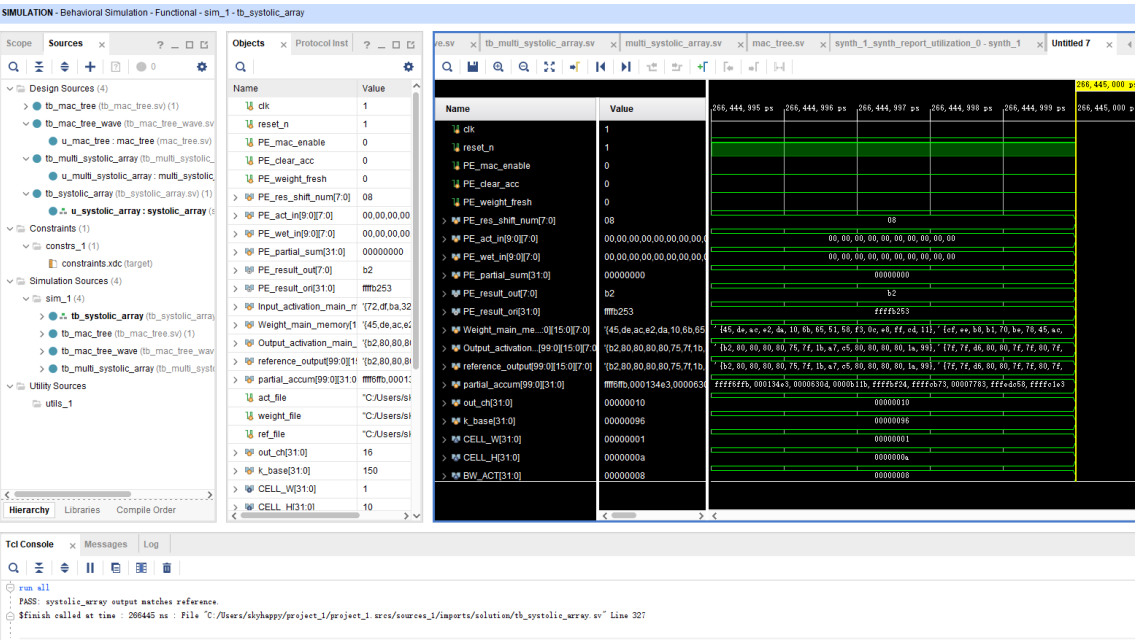


图 4: 1*10 脉动阵列仿真波形

```
1 systolic_array match_count=1600/1600, wrong_num=0
2 PASS: systolic_array all 1600 outputs match reference.
```

Listing 2: 1 × 10 脉动阵列仿真检查结果

1*10 脉动阵列的资源利用如下：

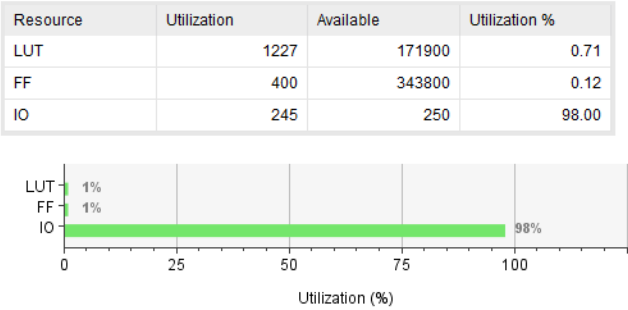


图 5: 1*10 脉动阵列资源利用报告

1*10 脉动阵列的时序报告如下：

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.727 ns	Worst Hold Slack (WHS): 0.249 ns	Worst Pulse Width Slack (WPWS): 4.650 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 320	Total Number of Endpoints: 320	Total Number of Endpoints: 401

图 6: 1*10 脉动阵列时序报告

通过时序报告可以看到，时序约束符合要求，并且在 100MHz 的时钟下，WNS 为 5.727ns，说明可以用更快的时钟运行，并且提升空间很高。

表 2: 1*10 脉动阵列在 100MHz 下的设计参数

算力	$1 \times 10 \times 100M = 1\text{Gops}$
计算量/算力	$100 \times 150 \times 16 / 1\text{Gops} = 240\text{us}$
最高时钟频率	$\frac{1}{10ns - 5.727ns} = 234.02\text{MHz}$
阵列利用率	$240 / 266.445 = 90.07\%$
IA 带宽需求	$8b \times 1 \times 10 \times 100\text{MHz} = 8\text{Gbps}$
W 带宽需求	$8b \times 10 \times 100\text{MHz} = 8\text{Gbps}$
OA 带宽需求	$32b \times 100\text{MHz} = 3.2\text{Gbps}$
W 片上 buffer 容量	$8b \times 10 = 80\text{bit}$
OA 片上 buffer 容量	32bit
IA 片外访问总 bit 数	$16 \times 100 \times 150 \times 8\text{bit}$
OA 片外写回总 bit 数	$16 \times 10 \times 150 \times 8\text{bit}$
W 片外访问总 bit 数	$16 \times 100 \times 8\text{bit}$

在 100MHz 时钟下, 该阵列每拍完成 10 个乘加, 理论峰值算力为 $10 \times 100\text{MHz} = 1\text{Gops}$ 。由于只有一个输出列, 输出写回峰值带宽为 $8b \times 100\text{MHz} = 800\text{Mbps}$, 低于参考 1×10 独立 MAC 阵列的 8Gbps 输出压力。

2.3 拓展脉动阵列为 $M \times N$ 设计与实现

本实验将上述 $1 \times N$ 脉动阵列扩展为 $M \times N$, 对应代码文件为 `multi_systolic_array.sv`, 测试文件为 `tb_multi_systolic_array.sv`。本次参数取 `CELL_H=10`、`CELL_W=4`, 一次并行计算 4 个输出通道, 每个输出通道对应一列 partial sum 流水链。由于本次输出通道数为 16, 刚好可以被 4 整除, 因此 testbench 每次处理 4 个输出通道, 循环 4 次即可覆盖全部输出通道。

拓展的关键是将 $1 \times N$ 脉动阵列中的部分和寄存器扩展为二维: `psum_reg[CELL_H][CELL_W]`。同一个激活值同时广播给 4 个输出列, 而每一列使用各自的权重和 partial sum, 因此可以复用 IA 读取并提升输出通道方向的并行度。对于 `CELL_W=4` 的情况, 每个 tile 加载 10×4 个权重, 随后流过 100 行输入, 同时更新 4 个输出通道的 100 个 partial sum。

该设计的前仿结果完全正确, 总耗时为 66645ns, 与上一设计时间开销缩小为 1/4, 符合预期。仿真结果如下:

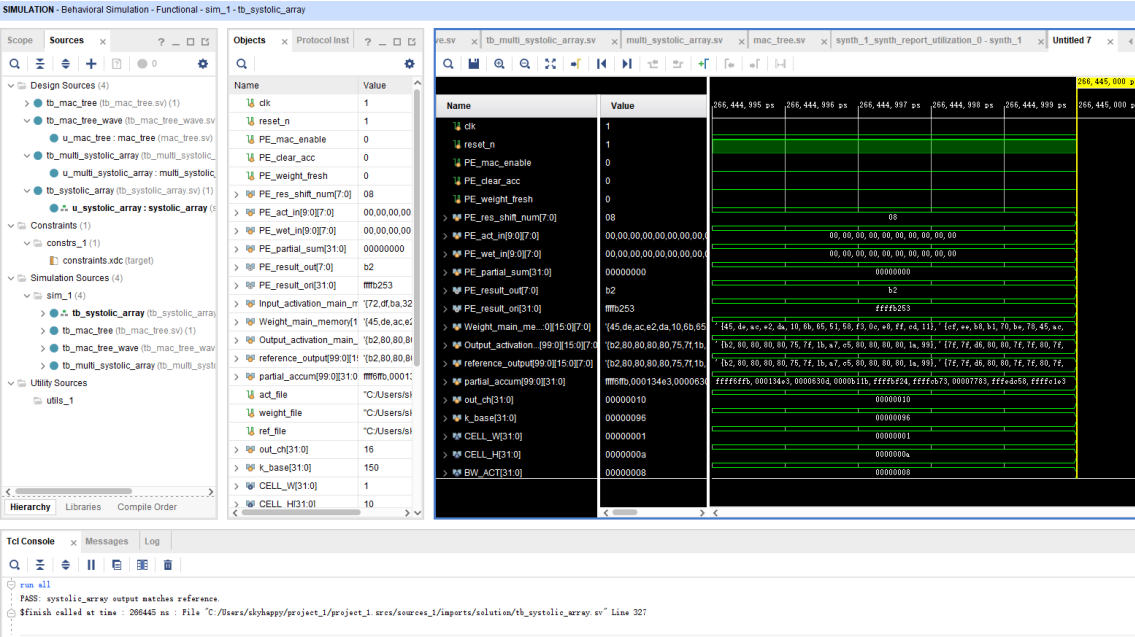


图 7: 4*10 脉动阵列仿真波形

```
1 multi_systolic_array match_count=1600/1600, wrong_num=0
2 PASS: multi_systolic_array all 1600 outputs match reference.
```

Listing 3: 4 × 10 脉动阵列仿真检查结果

1*10 脉动阵列的资源利用如下：

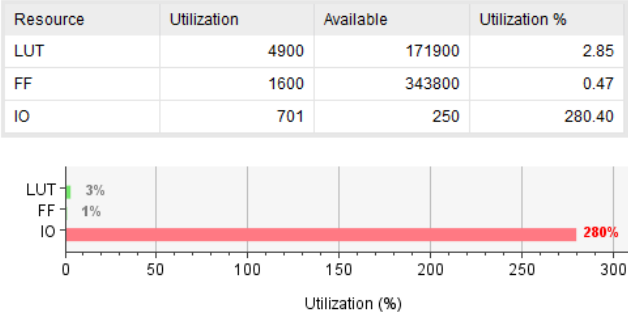


图 8: 4*10 脉动阵列资源利用报告

1*10 脉动阵列的时序报告如下：

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.727 ns	Worst Hold Slack (VHS): 0.249 ns	Worst Pulse Width Slack (WPWS): 4.650 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 1280	Total Number of Endpoints: 1280	Total Number of Endpoints: 1601

图 9: 4*10 脉动阵列时序报告

通过时序报告可以看到，时序约束符合要求，并且在 100MHz 的时钟下，WNS 为 5.727ns，说明可以用更快的时钟运行，并且提升空间很高。

表 3: 4*10 脉动阵列在 100MHz 下的设计参数

算力	$4 \times 10 \times 100M = 4Gops$
计算量/算力	$100 \times 150 \times 16 / 4Gops = 60us$
最高时钟频率	$\frac{1}{10ns - 5.727ns} = 234.02MHz$
阵列利用率	$60 / 66.645 = 90.03\%$
IA 带宽需求	$8b \times 1 \times 10 \times 100MHz = 8Gbps$
W 带宽需求	$8b \times 10 \times 100MHz \times 4 = 32Gbps$
OA 带宽需求	$32b \times 100MHz \times 4 = 12.8Gbps$
W 片上 buffer 容量	$8b \times 10 \times 4 = 320bit$
OA 片上 buffer 容量	$32b \times 4 = 128bit$
IA 片外访问总 bit 数	$16 \times 100 \times 150 \times 8bit$
OA 片外写回总 bit 数	$16 \times 10 \times 150 \times 8bit$
W 片外访问总 bit 数	$16 \times 100 \times 8bit$

在 100MHz 时钟下，该阵列每拍完成 $10 \times 4 = 40$ 个乘加。相对于 $1 \times N$ 脉动阵列，IA 带宽基本不变，但同一份输入激活值被 4 个输出通道共享。

从结果看，乘累加树适合在 K 维上展开并行，单次输出块的计算周期较少，但 IA 输入峰值带宽较高； $1 \times N$ 脉动阵列输出带宽压力较小，但输出通道并行度不足； $M \times N$ 脉动阵列在复用同一份 IA 的同时并行计算多个输出通道，是本实验中综合延时和带宽复用更均衡的方案。

3 思考题

1. 用非 GEMM 的方式算卷积（比如 Winograd 卷积，或者直接算卷积），该如何控制数据向阵列的传输？

对于 Winograd 卷积，输入特征图和卷积核都不能直接按普通 GEMM 的 K 维输入阵列，而是需要先按 tile 做变换。例如常见的 $F(2 \times 2, 3 \times 3)$ 会把一个输入小块变换到 Winograd 域，再与变换后的权重做逐点乘，最后做反变换得到输出。因此数据控制上应当增加一个 tile buffer：先从片外读入覆盖当前输出 tile 所需的输入区域，在片上完成 Winograd 输入变换，再将变换后的若干通道数据送入乘加阵列。权重也可以预先变换后缓存在片上，减少运行时重复变换。

对于直接卷积，可以不做 img2col 展开，而是在片上维护 line buffer 和滑动窗口 buffer。当卷积窗口向右移动时，大部分输入像素可以复用，只需要读入新进入窗口的一列；当窗口换行时，line buffer 更新一行数据。这样送入阵列的不是完整展开矩阵，而是窗口内的 $C \times R \times S$ 个激活值和对应权重。控制逻辑需要产生窗口坐标、通道坐标和 padding 判断，并将同一个输入像素广播到多个输出通道的 PE 中，提高输入复用率。

2. 在你之前设计的基础上，如果有更大的片上 buffer（或者有一个新的 shared buffer），你会怎么重新进行 mapping 的设计？会有什么样的好处？（提示：40nm 工艺下，外部访存的功耗是内部访存的 3~6 倍以上）可以进行一个 case study。

如果有更大的片上 buffer，我会优先改变权重和 partial sum 的 mapping，使其尽量留在片上。本次任务的权重矩阵大小为 $150 \times 16 \times 8bit = 19200bit$ ，约为 2.4KB，实际上很适合完整缓存在 shared buffer 中。若将全部权重一次性读入片上，则乘累加树 mapping 中权重不必对每个 10 行输出块重复从片外读取，W 片外访问量可以从 $16 \times 10 \times 150 \times 8bit$ 降到 $16 \times 150 \times 8bit$ ，约减少 10 倍。

对于 $M \times N$ 脉动阵列，还可以在 shared buffer 中保存若干输出行、若干输出通道的 32 位 partial sum。这样每个 K tile 结束后不必把 partial sum 写回片外，而是在片上等待下一个 K tile 继续累加，最终只写回 8 位量化后的 OA。由于片外访存功耗显著高于片内访存，这种 weight/partial-sum stationary 的 mapping 能同时降低能耗和延时。若 buffer 足够大，还可以采用 double buffer，在阵列计算当前 tile 时预取下一 tile 的 IA 和 W，隐藏数据搬运延迟。

3. 在你之前设计的基础上，如果实际不能满足你的带宽需求，比如参考代码的 OA 带宽需求是 8000Mbps，但实际只能做到 800Mbps，那该如何重新进行设计？可以进行一个 case study。

如果 OA 带宽只有 800Mbps，本质上等价于 100MHz 下每周期只能写回 1 个 8 位输出。对于原参考代码或本实验的乘累加树阵列，一次可能产生 10 个 8 位输出，峰值 OA 带宽为 8Gbps，不能直接写回片外。解决方法是增加片上 output buffer，将阵列一次产生的多个输出先写入片上 FIFO，再用 1 个 8 位端口串行写回片外。

以本实验的乘累加树为例，每个输出块得到 10 个输出。如果外部 OA 端口只有 800Mbps，则需要 10 个周期才能写完这 10 个结果。为了避免阵列完全停顿，可以使用双 output buffer：buffer A 串行写回上一组输出时，阵列继续计算下一组输出并写入 buffer B。当计算时间短于写回时间时，最终性能仍会被 OA 带宽限制；此时应减少输出并行度，或者改用 $1 \times N$ 脉动阵列这类每周期只产生 1 个输出的结构。本实验的 $1 \times N$ 脉动阵列输出写回峰值正好是 $8b \times 100\text{MHz} = 800\text{Mbps}$ ，因此在该带宽约束下更容易稳定运行，只是算力利用率较低。

4. 在你之前设计的基础上，如果阵列的尺寸并不是新任务中输入矩阵尺寸的因数，例如在参考代码中，阵列尺寸是 1×10 ，来了个新任务输入激活值矩阵尺寸是 102×128 ，权重矩阵尺寸是 128×18 。那么你该如何进行 mapping？或者如何进行更好的阵列设计？可以进行一个 case study。

若仍使用 1×10 阵列，可以采用分块加补零的方式完成 mapping。输入矩阵为 102×128 ，权重矩阵为 128×18 ，输出为 102×18 。行方向按 10 行切分时，前 10 个 tile 覆盖 100 行，最后一个 tile 只有 2 行有效，需要补 8 行零；输出通道方向按 1 个通道循环 18 次； $K = 128$ 若按 10 切分，则需要 13 个 K tile，最后一个 tile 只有 8 个有效元素，剩余 2 个补零。这样可以保证结果正确，但最后的行 tile 和最后的 K tile 都会降低阵列利用率。

更好的阵列设计应尽量让阵列参数贴合任务尺寸。例如可以选择输出通道并行度 $M = 6$ ，因为 18 可以被 6 整除； K 方向可以选择 8 或 16，因为 128 可以被 8 和 16 整除。对于脉动阵列，一个 8×6 或 16×6 的阵列都比 10×1 更适合该任务：权重 tile 不需要补零，输出通道也没有尾块浪费。若还希望行方向也整除，可以让乘累加树的 row lane 取 6 或 17 等能更好覆盖 102 的参数，或者采用可屏蔽 lane 的设计，在最后一个 tile 只使能有效输出行。

5. 在你之前设计的基础上，试着去搜索一下更好的 mapping 方案。（延时更低、带宽需求更低、访问 bit 数更低等）

更好的 mapping 通常围绕数据复用展开。本实验中比较直接的优化是采用 weight stationary 或 output stationary 的混合策略：权重矩阵较小，可以完整缓存在片上；partial sum 使用 32 位片上 buffer 保存，直到所有 K tile 累加结束后再量化写回。这样可以减少权重和中间结果的片外访问。对于卷积本身，如果不强制 GEMM 展开，还可以采用 row stationary mapping：输入特征图的相邻卷积窗口共享大量像素，使用 line buffer 保存多行输入，使每个输入像素在多个窗口和多个输出通道之间复用，从而降低 IA 访问 bit 数。

对于本实验的 $M \times N$ 脉动阵列，当前已经体现了一部分输入复用：同一个 $\text{PE_act_in}[i]$ 同时广播给 4 个输出通道列。若继续增大 CELL_W，例如从 4 增大到 8 或 16，可以进一步提高权重列方向并行度，并减少重复读取 IA 的次数。但 CELL_W 增大后，权重带宽和 partial sum

寄存器数量也会增加，因此需要结合片上 buffer 容量和外部带宽共同选择。

6. 试着优化你的设计，使其能够跑到更高的时钟/更短的任务时间。

要提高时钟频率，乘累加树设计的主要瓶颈是组合乘法和多级加法树路径较长。可以在加法树各层之间加入流水寄存器，或者使用 FPGA 的 DSP 乘法器内部寄存器，把一拍内的长组合路径拆成多拍流水。这样会增加输出延迟，但可以显著提高最高时钟频率；testbench 只需要相应增加结果采样延迟即可。

要缩短任务时间，可以从三个方向优化。第一，增大并行度，例如将 $M \times N$ 脉动阵列的 CELL_W 从 4 提高到 8 或 16，使一次处理更多输出通道。第二，减少控制气泡，本实验中每个 K tile 之间有加载权重、清空流水等控制周期，可以用双缓冲权重寄存器和 valid 信号让下一组权重预加载，从而让计算流更连续。第三，优化片上 buffer，使权重和 partial sum 尽量不访问片外，并通过 double buffer 重叠数据搬运和计算。综合来看，若资源允许，采用更宽的 $M \times N$ 脉动阵列并配合片上 weight buffer 和 partial sum buffer，是本任务中获得更短运行时间的主要方向。